

Other Synchronization Primitives

Lecture 12

Professor Amanda Bienz

Infinitely Many Synchronization Primitives

- **Java Monitors**
 - Monitors were originally in the Mesa language
 - Per-object locks and condition variables
 - Locks/unlocks on enter/leave a 'synchronized' method or block
 - Conditions occur with `o.wait()`, `o.notify()`, `o.notifyAll()`
- **Transactions - database-style**
 - “Begin transaction” and “end transaction” on data locations
 - Generally a set of database rows
 - Some processors support it on memory locations
- **Today : More about Semaphores and introducing Read-Write Locks**

Semaphore API in C

```
1  #include <semaphore.h>
2  sem_t s;
3  sem_init(&s, 0, 1); // initialize s to the value 1
```

- Include the semaphore header file
- Declare a semaphore s
- `sem_init(&s, 0, 1)` : initializes the semaphore to 1 and indicates that the semaphore is shared between **threads in the same process**
- Alternatively `sem_init(&s, 1, 1)` indicates semaphore is shared between processes with no shared memory

Wait

```
1  int sem_wait(sem_t *s) {  
2      decrement the value of semaphore s by one  
3      wait if value of semaphore s is negative  
4  }
```

- `sem_wait()`
 - If the value of the semaphore was one or higher when called `sem_wait()`, return right away
 - The caller will suspend execution waiting for a subsequent post
 - When negative, the value of the semaphore is equal to the number of waiting threads

Post (or Signal)

```
1  int sem_post(sem_t *s) {  
2      increment the value of semaphore s by one  
3      if there are one or more threads waiting, wake one  
4  }
```

- `sem_post()`
 - Simply increment the value of the semaphore
 - If there is a thread waiting to be woken, **wake** one of them up

Binary Semaphores (Locks)

```
1  sem_t m;  
2  sem_init(&m, 0, X); // initialize semaphore to X; what should X be?  
3  
4  sem_wait(&m);  
5  //critical section here  
6  sem_post(&m);
```

- What should X be?
 - The initial value should be 1

Thread Trace : Single Thread Using Semaphore

Value of Semaphore	Thread 0	Thread 1
1		
1	call sema_wait()	
0	sem_wait() returns	
0	(crit sect)	
0	call sem_post()	
1	sem_post() returns	

Thread Trace : Two Threads Using a Semaphore

Value	Thread 0	State	Thread 1	State
1		Running		Ready
1	call sem_wait()	Running		Ready
0	sem_wait() returns	Running		Ready
0	(crit set: begin)	Running		Ready
0	<i>Interrupt; Switch → T1</i>	Ready		Running
0		Ready	call sem_wait()	Running
-1		Ready	decrement sem	Running
-1		Ready	(sem < 0) → sleep	sleeping
-1		Running	<i>Switch → T0</i>	sleeping
-1	(crit sect: end)	Running		sleeping
-1	call sem_post()	Running		sleeping
0	increment sem	Running		sleeping
0	wake(T1)	Running		Ready
0	sem_post() returns	Running		Ready
0	<i>Interrupt; Switch → T1</i>	Ready		Running
0		Ready	sem_wait() returns	Running
0		Ready	(crit sect)	Running
0		Ready	call sem_post()	Running
1		Ready	sem_post() returns	Running

Semaphores as Condition Variables

```
1  sem_t s;
2
3  void *
4  child(void *arg) {
5      printf("child\n");
6      sem_post(&s); // signal here: child is done
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     sem_init(&s, 0, X); // what should X be?
13     printf("parent: begin\n");
14     pthread_t c;
15     pthread_create(&c, NULL, child, NULL);
16     sem_wait(&s); // wait here for child
17     printf("parent: end\n");
18     return 0;
19 }
```

A Parent Waiting For Its Child

parent: begin
child
parent: end

The execution result

Thread Trace : Parent Waiting for Child (Case 1)

- The parent call `sem_wait()` before the child has called `sem_post()`

Value	Parent	State	Child	State
0	Create(Child)	Running	<i>(Child exists; is runnable)</i>	Ready
0	call <code>sem_wait()</code>	Running		Ready
-1	decrement sem	Running		Ready
-1	$(sem < 0) \rightarrow \text{sleep}$	sleeping		Ready
-1	<i>Switch</i> →Child	sleeping	child runs	Running
-1		sleeping	call <code>sem_post()</code>	Running
0		sleeping	increment sem	Running
0		Ready	wake(Parent)	Running
0		Ready	<code>sem_post()</code> returns	Running
0		Ready	<i>Interrupt; Switch</i> →Parent	Ready
0	<code>sem_wait()</code> retruns	Running		Ready

Thread Trace : Parent Waiting for Child (Case 2)

- The child runs to completion before the parent call sem_wait()

Value	Parent	State	Child	State
0	Create (Child)	Running	<i>(Child exists; is runnable)</i>	Ready
0	<i>Interrupt; switch→Child</i>	Ready	child runs	Running
0		Ready	call sem_post()	Running
1		Ready	increment sem	Running
1		Ready	wake(nobody)	Running
1		Ready	sem_post() returns	Running
1	parent runs	Running	<i>Interrupt; Switch→Parent</i>	Ready
1	call sem_wait()	Running		Ready
0	decrement sem	Running		Ready
0	(sem<0)→awake	Running		Ready
0	sem_wait() retruns	Running		Ready

The Producer/Consumer (Bounded-Buffer) Problem

- Producer : put() interface
 - Wait for buffer to become empty in order to put data into it
- Consumer : get() interface
 - Wait for a buffer to become filled before using it

```
1  int buffer[MAX];
2  int fill = 0;
3  int use = 0;
4
5  void put(int value) {
6      buffer[fill] = value;    // line f1
7      fill = (fill + 1) % MAX; // line f2
8  }
9
10 int get() {
11     int tmp = buffer[use];    // line g1
12     use = (use + 1) % MAX;    // line g2
13     return tmp;
14 }
```

The Producer/Consumer (Bounded-Buffer) Problem

```
1  sem_t empty;
2  sem_t full;
3
4  void *producer(void *arg) {
5      int i;
6      for (i = 0; i < loops; i++) {
7          sem_wait(&empty);    // line P1
8          put(i);              // line P2
9          sem_post(&full);     // line P3
10     }
11 }
12
13 void *consumer(void *arg) {
14     int i, tmp = 0;
15     while (tmp != -1) {
16         sem_wait(&full);      // line C1
17         tmp = get();          // line C2
18         sem_post(&empty);     // line C3
19         printf("%d\n", tmp);
20     }
21 }
22 ...
```

First Attempt: Adding the Full and Empty Conditions

The Producer/Consumer (Bounded-Buffer) Problem

```
21  int main(int argc, char *argv[]) {  
22      // ...  
23      sem_init(&empty, 0, MAX); // MAX buffers are empty to begin with...  
24      sem_init(&full, 0, 0);    // ... and 0 are full  
25      // ...  
26  }
```

First Attempt: Adding the Full and Empty Conditions (Cont.)

- Imagine the MAX is greater than 1
 - If there are multiple producers, race condition can happen at line f1
 - It means that old data is overwritten
- We've forgotten **mutual exclusion**
 - The filling of a buffer and incrementing of the index into the buffer is a **critical section**

A Solution : Adding Mutual Exclusion

```
1  sem_t empty;
2  sem_t full;
3  sem_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          sem_wait(&mutex);      // line p0 (NEW LINE)
9          sem_wait(&empty);      // line p1
10         put(i);                // line p2
11         sem_post(&full);        // line p3
12         sem_post(&mutex);       // line p4 (NEW LINE)
13     }
14 }
15
(Cont.)
```

Adding Mutual Exclusion (Incorrectly)

A Solution : Adding Mutual Exclusion

```
(Cont.)
16  void *consumer(void *arg) {
17      int i;
18      for (i = 0; i < loops; i++) {
19          sem_wait(&mutex);          // line c0 (NEW LINE)
20          sem_wait(&full);           // line c1
21          int tmp = get();           // line c2
22          sem_post(&empty);          // line c3
23          sem_post(&mutex);          // line c4 (NEW LINE)
24          printf("%d\n", tmp);
25      }
26 }
```

Adding Mutual Exclusion (Incorrectly)

A Solution : Adding Mutual Exclusion (Cont.)

- Imagine two threads : one producer and one consumer
 - The consumer acquires the mutex (line c0)
 - The consumer calls `sem_wait()` on the full semaphore (line c1)
 - The consumer is blocked and yields the CPU
 - The consumer still holds the mutex!
 - The producer calls `sem_wait()` on the binary mutex semaphore (p0)
 - The producer is now stuck waiting too (**deadlock**)

Finally, A Working Solution

```
1  sem_t empty;
2  sem_t full;
3  sem_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          sem_wait(&empty);          // line p1
9          sem_wait(&mutex);          // line p1.5 (MOVED MUTEX HERE...)
10         put(i);                    // line p2
11         sem_post(&mutex);           // line p2.5 (... AND HERE)
12         sem_post(&full);            // line p3
13     }
14 }
15
(Cont.)
```

Adding Mutual Exclusion (Correctly)

Finally, A Working Solution

```
(Cont.)
16  void *consumer(void *arg) {
17      int i;
18      for (i = 0; i < loops; i++) {
19          sem_wait(&full);          // line c1
20          sem_wait(&mutex);         // line c1.5 (MOVED MUTEX HERE...)
21          int tmp = get();          // line c2
22          sem_post(&mutex);         // line c2.5 (... AND HERE)
23          sem_post(&empty);         // line c3
24          printf("%d\n", tmp);
25      }
26  }
27
28  int main(int argc, char *argv[]) {
29      // ...
30      sem_init(&empty, 0, MAX); // MAX buffers are empty to begin with ...
31      sem_init(&full, 0, 0);    // ... and 0 are full
32      sem_init(&mutex, 0, 1);   // mutex=1 because it is a lock
33      // ...
34  }
```


Semaphore vs Condition Variable

- Semaphores can act as condition variables
- What are the differences?
- Example : if buffer has 10 free spaces
 - Condition variable : wake up all consumers and if a consumer is in the first thread, it calls get()
 - Semaphore : Value of semaphore is 10; will be greater than 0 for first 10 threads

Reader-Writer Locks

- Imagine a number of concurrent list operations, include inserts and simple lookups
 - **Insert :**
 - Change state of list
 - Traditional critical section makes sense
 - **Lookup:**
 - Simply read the data structure
 - As long as we can guarantee no insert, we can allow many lookups to proceed concurrently

This special type of lock is known as a reader-write lock.

Reader-Writer Locks

- Only single writer can acquire the lock
- Once reader has a read lock:
 - More readers will be allowed to acquire read lock, also
- Writer will have to wait until all readers are finished

```
1  typedef struct _rwlock_t {
2      sem_t lock; // binary semaphore (basic lock)
3      sem_t writelock; // used to allow ONE writer or MANY readers
4      int readers; // count of readers reading in critical section
5  } rwlock_t;
6
7  void rwlock_init(rwlock_t *rw) {
8      rw->readers = 0;
9      sem_init(&rw->lock, 0, 1);
10     sem_init(&rw->writelock, 0, 1);
11 }
12
13 void rwlock_acquire_readlock(rwlock_t *rw) {
14     sem_wait(&rw->lock);
15     ...
```


Reader-Writer Locks (Cont.)

```
15     rw->readers++;
16     if (rw->readers == 1)
17         sem_wait(&rw->writelock); // first reader acquires writelock
18     sem_post(&rw->lock);
19 }
20
21 void rwlock_release_readlock(rwlock_t *rw) {
22     sem_wait(&rw->lock);
23     rw->readers--;
24     if (rw->readers == 0)
25         sem_post(&rw->writelock); // last reader releases writelock
26     sem_post(&rw->lock);
27 }
28
29 void rwlock_acquire_writelock(rwlock_t *rw) {
30     sem_wait(&rw->writelock);
31 }
32
33 void rwlock_release_writelock(rwlock_t *rw) {
34     sem_post(&rw->writelock);
35 }
```

A Reader-Writer Lock (Cont.)

- The reader-write locks have fairness problem
 - It would be relatively easy for the reader to start the writer
 - How to prevent more readers from entering the lock once a writer is waiting?